

유저 프로필 API 안내

이번에 프로필 관련 API를 새로 추가하고, 기존 "내 정보 조회"를 바꿨습니다. 바뀐 부분 위주로 정리했어요.

먼저 — 기존 코드 한 군데 고쳐주세요.

예전에 쓰던 `GET /api/users/me` 가 `GET /api/users/profile` 로 바뀌었습니다. 하는 일은 똑같으니 **주소만 바꿔주면** 됩니다. 응답에 프로필 이미지(`profileImage`) 한 칸이 더 생긴 것 빼고는 나머지(닉네임·이메일 등)랑 토 큰 쓰는 방식은 그대로예요.

1. 내 프로필 조회 `GET /api/users/profile`

로그인한 사람 본인 정보를 가져옵니다. (기존 `/me` 자리)

응답 `data` 안에 이런 값들이 들어옵니다:

필드	설명
<code>userId</code>	유저 번호
<code>email</code>	이메일
<code>nickname</code>	닉네임
<code>role / provider</code>	권한, 가입 경로 (기존과 동일)
<code>profileImage</code>	프로필 사진 주소. 이번에 새로 추가됨. 설정 안 했으면 <code>null</code> 이라 기본 이미지로 처리하면 됩니다.

```
const res = await fetch("/api/users/profile", {
  headers: { Authorization: `Bearer ${accessToken}` },
});
const { data } = await res.json();
// data.profileImage 가 null 이면 기본 이미지 사용
```

2. 프로필 수정 `PATCH /api/users/profile`

닉네임이랑 프로필 사진을 바꿉니다. **바꾸고 싶은 것만 보내면** 되고, 안 넣은 건 그대로 둡니다. 둘 다 보내서 한 번에 바뀌도 됩니다.

```
// 닉네임만 바꾸는 경우
await fetch("/api/users/profile", {
  method: "PATCH",
  headers: {
    Authorization: `Bearer ${accessToken}`,
    "Content-Type": "application/json",
```

```
},
body: JSON.stringify({ nickname: "새닉네임" }),
});
```

보낼 수 있는 값은 `nickname`, `profileImageKey` 두 개입니다. (`profileImageKey` 가 뭔지는 아래 4번에서 설명할 게요.)

응답은 1번 조회랑 똑같은 형태로, 수정된 최신 정보가 옵니다.

이럴 땐 막힙니다:

- 닉네임이 비어있거나 공백이면 → `400 (INVALID_INPUT)`
- 이미 다른 사람이 쓰는 닉네임이면 → `409 (DUPLICATE_NICKNAME)`. 내가 원래 쓰던 닉네임 그대로 보내는 건 괜찮습니다.

3. 회원 탈퇴 `DELETE /api/users`

로그인한 계정을 탈퇴시킵니다. 주소가 `/api/users/profile` 이 아니라 그냥 `/api/users` 인 점만 주의하세요.

```
await fetch("/api/users", {
  method: "DELETE",
  headers: { Authorization: `Bearer ${accessToken}` },
});
```

성공하면 끝입니다. 탈퇴 뒤엔 그 토큰이 무효가 되니, 저장해둔 토큰·로그인 상태를 비우고 로그인 화면으로 보내주세요.

4. 프로필 사진 바꾸기 (3단계)

사진은 우리 서버를 거치지 않고 **프론트에서 S3로 바로 올립니다**. 순서는 이렇습니다.

1. 업로드할 주소를 받아온다 — `GET /api/users/profile/image/presigned-url?`

`filename=avatar.png&contentType=image/png`

파일 이름이랑 형식을 넘기면 `presignedUrl` 과 `s3Key` 를 줍니다. 형식은 `jpeg / png / webp / gif` 만 됩니다.

2. 받은 `presignedUrl` 로 사진 파일을 직접 올린다 (PUT)

이 요청은 S3로 바로 가는 거라 우리 토큰(Authorization)은 넣으면 안 됩니다.

3. 받았던 `s3Key` 를 2번 프로필 수정으로 넘긴다 — `{ profileImageKey: s3Key }`

그럼 서버가 사진을 확정 저장하고, 응답 `profileImage` 에 최종 주소를 돌려줍니다.

```
// 1) 올릴 주소 받기
const q = new URLSearchParams({ filename: file.name, contentType: file.type });
const r1 = await fetch(`/api/users/profile/image/presigned-url?${q}`, {
  headers: { Authorization: `Bearer ${accessToken}` },
});
const { data } = await r1.json();
const { presignedUrl, s3Key } = data;

// 2) S3에 파일 직접 업로드 (토큰 넣지 않기)
```

```
await fetch(presignedUrl, {
  method: "PUT",
  headers: { "Content-Type": file.type },
  body: file,
});

// 3) 프로필에 반영
const r3 = await fetch("/api/users/profile", {
  method: "PATCH",
  headers: {
    Authorization: `Bearer ${accessToken}`,
    "Content-Type": "application/json",
  },
  body: JSON.stringify({ profileImageKey: s3Key }),
});
const updated = await r3.json();
// updated.data.profileImage 가 최종 사진 주소
```

더 자세한 명세나 다른 에러 케이스는 Swagger([/swagger-ui.html](#))에서 볼 수 있어요. 막히면 편하게 물어보세요.